

For integration metadata, the date of issue is defined as the date of the first Bitcoin BIP110 branch block at height 961632. The circulating-supply-at-issuance figure is the nominal subsidy-derived supply through the last common height 961631. It does not subtract coins that were lost, burned, or otherwise provably unspendable.

Authoritative implementation references are listed at the end of this guide.

Branding and logo assets

Use the project-provided Bitcoin logo assets for exchange listings, wallet interfaces, custody systems, and integration pages.

Priority	Asset	URL	Usage
First choice	Orange logo	https://bitcoinbip110.org/downloads/logo-orange-on-transparent-bg.svg	Preferred Bitcoin BIP110 logo.
Second choice	Black logo	https://bitcoinbip110.org/downloads/logo-black-on-transparent-bg.svg	Use when the orange logo cannot be used.
	Black logo with white contour	https://bitcoinbip110.org/downloads/logo-black-on-transparent-bg-with-contour.svg	Use the white-contour version when the black logo is presented on a dark background.

The Resource Directory also shows the contoured black logo as the dark-background example. The example uses the same contoured SVG asset.

What the exchange owns and what the customer owns

At the chain split, every unspent output in the shared ledger existed on both resulting chains. If an exchange controlled the private keys for those outputs, the exchange controlled the corresponding outputs on both chains.

A customer's displayed BTC balance on a custodial exchange is an internal liability of the exchange. The protocol does not automatically create a customer BIP110 balance inside the exchange database. The exchange must define a credit policy, reconstruct eligible customer balances, recover the corresponding BIP110 assets, and reconcile assets against liabilities before crediting customers.

The shared on-chain state is the UTXO set after block 961631. The exchange should document how it treats internal ledger events that were pending around the split, including deposits, withdrawals, internal transfers, margin positions, loans, earn products, and third-party custody.

Phase 1: Define the exchange policy

Step 1. Choose the support level

An exchange can support Bitcoin BIP110 in stages.

Distribution only

- Reconcile fork assets
- Credit eligible customer balances
- Enable withdrawals
- Keep deposits disabled
- Keep trading disabled

Custody support

- Credit eligible balances
- Enable deposits
- Enable withdrawals
- Operate hot and cold wallets
- Keep trading optional

Full listing

- Deposits
- Withdrawals
- Trading pairs
- Market surveillance
- Market making or liquidity support as applicable

Supporting fork distribution does not require the exchange to open a market.

Step 2. Define the customer snapshot policy

Use the shared chain state through height 961631 as the technical fork reference.

Define in writing:

- the internal ledger cutoff used for customer balances;
- treatment of deposits confirmed on the shared chain before the split;
- treatment of deposits that were pending or credited provisionally;
- treatment of withdrawals requested but not yet broadcast;
- treatment of withdrawals broadcast after the split;
- treatment of balances held in margin, lending, earn, institutional custody, or subaccounts;
- treatment of BTC held through an external custodian;
- minimum creditable amount and any dust policy;
- whether credits are 1:1 at the satoshi level for qualifying liabilities.

Use integer base units. One coin equals 100,000,000 base units.

Step 3. Freeze fork-sensitive operations during reconciliation

Before moving shared pre-fork UTXOs, stop any workflow that could create ambiguous cross-chain spends.

Recommended temporary controls:

- pause BIP110 deposits and withdrawals until the BIP110 node is verified;
 - prevent automated wallet sweeps from spending unsplit pre-fork UTXOs;
 - prevent BTC and BIP110 systems from sharing the same hot-wallet process;
 - preserve transaction logs from the split onward for replay analysis.
-

Phase 2: Deploy and verify the BIP110 node

Step 4. Install the reference release

Use Bitcoin Knots 29.4.1.knots20260508 or a later release that explicitly supports the same BLAKE2b mainnet chain.

Official release directory:

<https://bitcoinknots.org/files/29.x/29.4.1.knots20260508/>

Verify the downloaded build against the published `SHA256SUMS` and `SHA256SUMS.asc` before deployment.

Step 5. Isolate BIP110 infrastructure from BTC infrastructure

Bitcoin BIP110 retains several Bitcoin mainnet identifiers. Do not rely on network name, address prefix, genesis block, or default port as a unique chain identifier.

Use:

- a dedicated data directory;
- a dedicated process or container;
- a dedicated wallet database or external signer namespace;
- separate monitoring and alerting;
- separate internal asset identifiers;
- separate hot and cold wallet descriptors;
- preferably separate hosts or virtual machines for BTC and BIP110.

Mainnet defaults in the reference release are P2P port 8333 and RPC port 8332. If BTC and BIP110 are hosted on the same machine, override ports to prevent collisions. Keep RPC private.

Step 6. Synchronize the node

Allow the node to synchronize beyond height 961640.

If an older node followed the post-split SHA256d branch, the current release may walk back and follow the BLAKE2b chain. A pruned node that lacks required historical data may need a full resynchronization. Follow the current Knots release notes for recovery behavior.

Step 7. Perform deterministic chain-identity checks

The following checks should be mandatory at service startup and before enabling deposits or withdrawals:

```
bitcoin-cli getblockhash 961632
bitcoin-cli getblockhash 961639
bitcoin-cli getblockhash 961640
```

Required values:

```
961632  00000000000000000000000000000000169eb6f811ddbd0daf343af7b62180cdb13e7c78dbc16
961639  000000000000000000000000000000001bbc439e13f749dca850d32c7a2834165338713027e65
961640  0000000000000000000000000000000050c1e5f69672f459293be14f46e5a494e7a8c8541396f18eeb
```

Then run:

```
bitcoin-cli getblockchaininfo
bitcoin-cli getdeploymentinfo
```

For a synchronized BIP110 mainnet node, verify that BLAKE2b is active at height 961640. During the temporary reduced-data window, verify that the reduced-data deployment is active at the same height.

Do not use `chain=main` as a unique identity check. Both chains can report mainnet semantics.

Step 8. Add a fail-closed chain health gate

Your exchange service should disable BIP110 deposits and withdrawals automatically if any of the following occurs:

- one of the required checkpoint hashes does not match;
- the node reports initial block download;
- the node falls materially behind its peers or independent monitoring;
- an unexpected deep reorganization occurs;
- the wallet or indexer falls behind the node;
- RPC is unavailable or returns inconsistent chain identity;
- the signer cannot produce the required transaction format for the UTXOs being spent.

The protocol does not specify an exchange confirmation count. Set deposit and withdrawal confirmation policies according to the exchange's own chain-risk model and current network conditions.

Phase 3: Recover and reconcile the exchange's BIP110 assets

Step 9. Inventory exchange-controlled pre-fork keys and descriptors

Identify every wallet, descriptor, xpub, HSM account, MPC vault, and third-party custody account that held BTC before the split.

For each custody domain record:

- wallet or vault identifier;
- key ownership or signing authority;
- pre-fork UTXOs;
- value in base units;
- whether the output has been spent on BTC since the split;
- whether the corresponding spend appeared on BIP110;
- whether the BIP110 output remains unspent;
- whether the output has already been separated.

Step 10. Rescan the BIP110 chain

Import watch-only descriptors or equivalent address metadata into the BIP110 custody system and rescan the chain as required.

The objective is to identify the BIP110 UTXOs currently controlled by the exchange, including pre-fork outputs that remain shared and any BIP110-specific descendants created after the split.

Step 11. Reconcile post-split replay activity

Ordinary signatures can remain valid on both chains when they spend the same pre-fork outputs. Review every BTC wallet transaction from the split onward and determine whether an equivalent transaction was also confirmed on BIP110.

Classify each pre-fork output as:

- unspent on both chains;
- spent only on BTC;
- spent only on BIP110;
- spent on both chains by replay;
- already separated into chain-specific descendants.

Do not assume that the exchange's current BTC UTXO inventory equals its current BIP110 UTXO inventory.

Step 12. Reconcile assets against customer liabilities

Build two independent totals:

BIP110 assets

Total BIP110 base units controlled by the exchange after replay analysis and custody recovery.

BIP110 customer liabilities

Total base units the exchange intends to credit under its published fork policy.

Require:

```
verified BIP110 assets >= customer BIP110 liabilities + operational reserves
```

Resolve any shortfall before customer credits become withdrawable.

Phase 4: Separate shared coins safely

Step 13. Create new BIP110-only operational wallets

Create new BIP110 hot and cold wallets after the fork.

Recommended controls:

- separate seeds or HSM namespaces from BTC;
- separate descriptors and derivation records;
- separate address databases;
- separate signing policies;
- separate withdrawal queues;
- separate accounting asset IDs.

Do not reuse BTC deposit addresses for BIP110 operations even though the address encodings are compatible.

Step 14. Implement or verify SIGHASH_UNIFIED support

`SIGHASH_UNIFIED` is an opt-in signature hash implemented by the current Knots release.

Key properties:

- hash-type bit: `0x20` ;
- tagged hash: `UnifiedSighash` ;
- applies to bare/P2SH, SegWit v0, Taproot key path, and Tapscript;
- replay protection is one-way and applies only to signatures that opt in;

- ordinary signatures remain valid and can remain replayable;
- external signers require the spent output information for every input;
- PSBT already has fields capable of carrying the required previous-output information;
- the reference documentation includes 166 test vectors.

For HSM, MPC, or hardware signing systems, verify support before using unsplit pre-fork UTXOs.

Step 15. Sweep unsplit BIP110 outputs with protected signatures

For each shared pre-fork UTXO that the exchange intends to use on BIP110:

1. Construct a BIP110 send-to-self transaction.
2. Send the output to a newly created BIP110-only operational wallet.
3. Sign each relevant input using `SIGHASH_UNIFIED`.
4. Broadcast only through the verified BIP110 node.
5. Wait for the exchange's required confirmation threshold.
6. Confirm that the transaction is present on BIP110.
7. Confirm that the corresponding BTC UTXO remains unspent on BTC.
8. Mark the resulting BIP110 outputs as chain-separated.

Do not reopen BIP110 withdrawals from pre-fork inventory until the exchange can distinguish separated outputs from shared outputs.

Step 16. Enforce a custody invariant

Production withdrawal selection should use only BIP110-specific outputs or outputs whose separation status is known.

A useful internal field is:

```
chain_separation_status = SHARED | BIP110_ONLY | BTC_ONLY | UNKNOWN
```

Prevent `SHARED` and `UNKNOWN` UTXOs from entering normal withdrawal coin selection.

Phase 5: Integrate the wallet and signer stack

Step 17. Choose the integration boundary

Two common designs are supported.

RPC-first design

The exchange talks to a BIP110-compatible Knots node and delegates transaction parsing and chain validation

to the node.

This minimizes custom handling of the new block-header format.

Native parser design

The exchange parses blocks or headers directly.

From height 961640, raw block-header consumers must support the 164-byte version 2 BLAKE2b header. Software that assumes an 80-byte SHA256d header will not validate this chain correctly.

Step 18. Update raw-header and block parsers

The reference node exposes additional version 2 header fields through `getblockheader` and `getblock`, including:

- `txcount`
- `header_version`
- `nonce2`
- `nonce3`
- `extranonce`
- `time_offset`
- `header_flags`
- `xor_key`
- `xor_key_mask_clear_bits`
- `mm_rhs`

If your exchange only consumes normalized RPC output and does not independently hash raw headers, the required change can be smaller. Independently validating header consumers must implement the BLAKE2b header rules from the reference implementation.

Step 19. Update address handling

Current mainnet address encodings overlap Bitcoin BTC:

- P2PKH: version byte `0x00`, commonly `1...`
- P2SH: version byte `0x05`, commonly `3...`
- Bech32 HRP: `bc`, including `bc1q...` and `bc1p...`

An address string therefore does not uniquely identify BTC or BIP110.

Exchange requirements:

- display an explicit network selector;
- label every deposit and withdrawal as Bitcoin BIP110 / BIP110;
- use a separate deposit-address database;
- never infer network solely from the address string;

- build an operational recovery procedure for accidental cross-chain deposits if the exchange chooses to support recovery.

Step 20. Preserve exact Bitcoin-style amount accounting

- 8 decimal places
 - 100,000,000 base units per coin
 - integer base-unit accounting internally
 - no floating-point balance arithmetic
-

Phase 6: Customer crediting

Step 21. Generate the fork-credit ledger

For each eligible account, calculate the BIP110 credit under the exchange's published policy.

Recommended audit record:

```
customer_id
snapshot_btc_base_units
included_pending_deposits
excluded_pending_deposits
included_pending_withdrawals
excluded_pending_withdrawals
product_adjustments
bip110_credit_base_units
policy_version
calculation_timestamp
review_status
```

Keep the source BTC ledger snapshot immutable after approval.

Step 22. Perform independent reconciliation

Before making balances withdrawable, have a second system or team verify:

- total customer credits;
- total BIP110 custody assets;
- replay adjustments;
- third-party custodian balances;
- hot and cold wallet allocations;
- any reserved operational balance.

Step 23. Credit balances

Credit the exchange's internal BIP110 asset ledger.

Publish the eligibility rule and snapshot methodology clearly enough that customers can understand whether they received a credit and why.

Phase 7: Deposits

Step 24. Create dedicated BIP110 deposit addresses

Generate deposit addresses from BIP110-only wallet infrastructure.

Do not reuse an existing customer's BTC deposit address as their BIP110 deposit address.

Step 25. Detect deposits through the verified BIP110 node

Credit only transactions observed on the verified BIP110 chain.

Before crediting, confirm:

- node chain anchors pass;
- node is synchronized;
- transaction is in a valid BIP110 block;
- required confirmation policy is satisfied;
- transaction has not been credited previously;
- destination address maps to the correct BIP110 account.

Step 26. Display cross-network warnings

Because BTC and BIP110 addresses can look identical, the deposit UI should state plainly:

"Send Bitcoin BIP110 on the BIP110 network only. BTC and BIP110 use compatible-looking address formats, so the address string does not identify the network."

Phase 8: Withdrawals

Step 27. Build withdrawals from separated BIP110 inventory

Normal withdrawals should spend only BIP110-specific UTXOs.

If an unsplit pre-fork UTXO must be used, the signer must produce a `SIGHASH_UNIFIED` protected spend and the transaction should first be used for separation rather than sent directly to a customer.

Step 28. Validate the withdrawal before broadcast

Check:

- asset ID is BIP110;
- destination was submitted under the BIP110 network selector;
- selected UTXOs are `BIP110_ONLY` ;
- node chain anchors pass;
- node is synchronized;
- fee is computed in BIP110 base units;
- signer policy passed;
- raw transaction decodes as expected on the BIP110 node.

Step 29. Broadcast through BIP110 infrastructure only

Submit the transaction to the verified BIP110 node and monitor confirmation there.

Do not use a shared BTC broadcast service for BIP110 withdrawals.

Phase 9: Testing and certification

Step 30. Test node identity

Test positive and negative cases:

- correct BIP110 node passes all three fork anchors;
- BTC node fails the 961632 identity check;
- stale or unsynchronized node keeps services disabled.

Step 31. Test replay-safe separation

Using controlled test funds:

1. identify an unsplit output;
2. create a `SIGHASH_UNIFIED` BIP110 send-to-self transaction;
3. confirm it on BIP110;
4. verify the equivalent transaction is invalid on a chain that does not implement the new signature hash;
5. verify the BTC-side source output remains available.

Step 32. Test external signer compatibility

For each supported script type, test:

- P2PKH or bare/P2SH where applicable;
- SegWit v0;
- Taproot key path if supported;
- Tapscript if supported;
- PSBT round trip;
- multi-input transactions with all spent-output data supplied;
- rejection when required previous-output data is missing.

Use the upstream unified-sighash test vectors for implementation validation.

Step 33. Test header and indexer compatibility

If the exchange has custom header or block parsing, test both sides of the transition:

- height 961639, SHA256d, legacy 80-byte header;
- height 961640, BLAKE2b, version 2, 164-byte header.

Step 34. Test accounting edge cases

Test:

- pending deposits;
- pending withdrawals;
- replayed post-split withdrawals;
- internal transfers;
- institutional subaccounts;
- negative or borrowed balances;
- third-party custody;
- dust amounts;
- duplicated credit jobs;
- rollback and re-run of the credit calculation.

Step 35. Test service fail-closed behavior

Simulate:

- checkpoint mismatch;
- RPC outage;
- stale tip;
- wallet lag;
- signer outage;
- unexpected reorganization.

Deposits and withdrawals should stop safely without changing already confirmed customer balances.

Phase 10: Go live

Step 36. Production readiness checklist

Before enabling customer access, confirm all of the following:

- ☐ Fork-credit policy approved
- ☐ Customer snapshot approved
- ☐ Bitcoin Knots 29.4.1 or later deployed
- ☐ Release hashes and signatures verified
- ☐ Separate BIP110 data directory and wallet infrastructure
- ☐ Height 961632 hash verified
- ☐ Height 961639 hash verified
- ☐ Height 961640 hash verified
- ☐ BLAKE2b activation verified at 961640
- ☐ Replay analysis completed
- ☐ BIP110 assets reconciled against liabilities
- ☐ Shared UTXOs separated or quarantined
- ☐ SIGHASH_UNIFIED signer support tested where required
- ☐ 164-byte header support tested where required
- ☐ Separate BIP110 deposit address system active
- ☐ Separate BIP110 withdrawal system active
- ☐ Chain health gate active
- ☐ Deposit confirmations policy approved
- ☐ Withdrawal confirmations policy approved
- ☐ Customer network warnings published
- ☐ Incident pause procedure tested
- ☐ Monitoring and alerting active

Step 37. Launch in controlled stages

Recommended sequence:

1. Make customer fork credits visible but locked.
2. Enable BIP110 withdrawals.
3. Observe wallet and chain behavior.
4. Enable BIP110 deposits.
5. Observe deposit accounting and reorg handling.
6. Enable trading only if the exchange has separately approved market support.

This staged rollout limits the number of systems that change at one time.

Ongoing monitoring

Monitor:

- node version;
- chain tip height and hash;
- peer count and synchronization state;
- fork anchor checks;
- deployment state;
- reorganization depth;
- wallet scan height;
- deposit indexer lag;
- withdrawal queue age;
- hot-wallet balance;
- cold-wallet reconciliation;
- count and value of `SHARED` UTXOs;
- signer support and error rate;
- RPC latency and failure rate.

Pause deposits and withdrawals when chain identity or wallet state becomes uncertain.

References

1. Bitcoin BIP110 Resource Directory: <https://bitcoinbip110.org/directory/#overview>
2. Developer guidance: <https://bitcoinbip110.org/developers>
3. Exchange integration: <https://bitcoinbip110.org/exchanges/>
4. Technical specifications: <https://bitcoinbip110.org/technicals/>